

SUGGESTED APPROACHES & METHODOLOGY

Predictive Modelling of Chronic Disease Prevalence:

An R-Based Analysis of Regional Health Disparities in India

Prepared for: National Ministry of Education Case Study Project

Software: R Programming (FLOSS)

Date: 11 May 2026

Section	Title	Key Topics
Section 1	Exploration & Pre-processing Techniques	Data loading, inspection, imputation, transformation, EDA
Section 2	Modelling, Diagnostics & Statistical Tests	ARIMA, TAR-2, TAR-3, ADF, ACF/PACF, RMSE, AIC
Section 3	R Packages & Pre-defined Functions	forecast, tsDyn, tseries, zoo, ggplot2, Metrics, knitr

1. Exploration and Pre-processing Techniques

The information is imported using base-R `read.csv()` and a relative file path, which makes sure that it can be used on any computer. Right after loading, the methods `head()`, `str()`, and `summary()` are used to check column types, find problems with the structure, and make sure the expected dimensions (1,344 rows x 7 columns) are met.

1.1. Data Loading and Initial Inspection

Base-R `read.csv()` is used to load the dataset with a relative file path, which makes sure that it can be used on any operating system. Right after the file is loaded, the methods `head()`, `str()`, and `summary()` are used to check the column types, look for structural problems, and make sure the expected dimensions (1,344 rows x 7 columns) are met.

```
# Step 1: Data Loading
df <- read.csv(file.path("data_directory",
  "CDPI_India_2021_2024.csv"), stringsAsFactors = FALSE)
head(df)
str(df)
summary(df)
```

What to look for: Note that Date should be read as a Date class and not a character. Also, make sure that CDPI, LiteracyRate, HealthIndex, and DALYs_NCD are all numbers and that State and Regime_TAR2 are character/factor. As the next step goes by, any mistakes are fixed.

1.2. Type Conversion and Date Parsing

For downstream models to work, the columns must be typed correctly. Date fields need to be as-parsed to R's Date class `Date()` so that functions that work with time series can understand them properly. It is possible to import numeric columns as characters if the CSV has any values that are not numbers; explicit coercion with `as.numeric()` checks for mistakes that don't show up.

```
# Step 2: Type conversion
df$Date <- as.Date(df$Date, format = "%Y-%m-%d")
df$CDPI <- as.numeric(df$CDPI)
df$LiteracyRate <- as.numeric(df$LiteracyRate)
df$HealthIndex <- as.numeric(df$HealthIndex)
df$DALYs_NCD <- as.numeric(df$DALYs_NCD)
```

1.3. Missing Value Detection and Imputation

There are 42 missing CDPI values in the dataset, which is 3.1% of all observations. This is because of delays in official reporting in three high-burden states. Use `is.na()` and `colSums(is.na(df))` to find missing values. Then, use the `na.approx()` function from the `zoo` package to fill them in with linear interpolation.

Method (R function)	Description	Rationale / Trade-off
Linear Interpolation (<code>na.approx</code> , <code>zoo</code>)	Fills in NA with the number between nearest neighbours that is linearly weighted.	Maintains a smooth temporal trend; little bias for short gaps in health series that change slowly

Last Observation Passed On (na.locf, zoo)	It replaces NA with the most recent number that is not NA.	Simple, but can cause step artefacts; not as good for rising series
Interpolation Based on Splines (na.spline, zoo)	Fits a cubic spline through numbers in the area	Curves that are smoother; may overshoot during uncertain times

Table 1.1: Comparison of imputation methods. Linear interpolation is used in this study.

Step 3: Detect and impute missing values

```
cat('Missing values per column:'); print(colSums(is.na(df)))
```

```
library(zoo)
```

```
df$CDPI <- na.approx(df$CDPI, rule = 2) cat('Remaining NAs:',  
sum(is.na(df$CDPI)))
```

1.4. Log Transformation

The statistics on the prevalence of chronic diseases is positively skewed, which means that a few states with a lot of cases pull the right tail. On the CDPI column, a natural logarithm change is used to:

- Stabilise variance across the time period (homoscedasticity)
- Linearise multiplicative growth patterns in NCD burden
- Bring the marginal distribution closer to normality, satisfying the implicit assumptions of ARIMA residual distributions
- Enable direct comparison with analogous log-price series used in financial time-series literature (Dubey, 2024)

Step 4: Apply natural log transformation

```
df$log_CDPI <- log(df$CDPI)
```

Verify descriptive statistics match the report (Table 3)

```
cat('Mean log(CDPI) : ', round(mean(df$log_CDPI), 4))
```

```
cat('Variance log(CDPI) : ', round(var(df$log_CDPI), 4))
```

```
cat('SD log(CDPI) : ', round(sd(df$log_CDPI), 4))
```

1.5. Temporal Subsetting and Aggregation

The study window is restricted to **January 2021 – December 2024** to focus on post-COVID-19 NCD dynamics. For time-series modelling, the 28-state panel is collapsed to a **national monthly average series** (48 time points) using `dplyr::group_by()` and `summarise()`. This national series is then converted to an R *ts* object for use with *forecast* and *tsDyn* modelling functions.

```
# Step 5: Subset and aggregate to national monthly average
```

```
library(dplyr)
df <- df[df$Date >= as.Date('2021-01-01'), ]
national_df <- df |>
group_by(Date) |>
summarise(log_CDPI_mean = mean(log_CDPI), .groups = 'drop') |>
arrange(Date)
# Convert to ts object (monthly frequency =
12) national_ts <-
ts(national_df$log_CDPI_mean, start = c(2021,
1), frequency = 12)
```

1.6. Exploratory Data Analysis (EDA)

Three families of visualisation are produced before model specification to characterise the data's temporal structure, distributional properties, and socioeconomic relationships.

Visualisation	R Code Pattern	Analytical Purpose
Time Series Plot	<code>ggplot2::ggplot() + geom_line()</code>	Reveal trend, seasonality, and regime-switching structure across state clusters
ACF Plot	<code>acf()</code>	Quantify lag-wise autocorrelation; slow decay => non-stationarity (supports $d=1$)
PACF Plot	<code>pacf()</code>	Isolate direct lag effects; significant spike at lag 1 => AR(1) specification
Correlation Heatmap	<code>corrplot::corrplot(cor(...))</code>	Map pairwise relationships between CDPI, literacy, health index, and DALYs
Box Plot by Regime	<code>ggplot2::geom_boxplot()</code>	Compare CDPI distributions between High- and Low-burden TAR regimes
Scatter: CDPI vs Literacy	<code>ggplot2::geom_point() + geom_smooth(method=' lm')</code>	Visualise the negative socioeconomic gradient in chronic disease burden

Table 1.2: EDA visualisations applied to the CDPI dataset.

```
# Step 6: EDA — time series, ACF/PACF,
correlation library(ggplot2); library(corrplot)
# Time series plot for all states
ggplot(df, aes(x=Date, y=log_CDPI, colour=Regime_TAR2,
group=State)) + geom_line(linewidth=0.6, alpha=0.75) +
labs(title='log(CDPI) -- India 2021-2024', x='Date',
y='log(CDPI)') # ACF and PACF side-by-side
```

```

par(mfrow=c(1,2))
acf(national_ts, main='ACF -- log(CDPI) National Average')
pacf(national_ts, main='PACF -- log(CDPI) National Average')
par(mfrow=c(1,1))
# Correlation heatmap
corrplot(cor(df[,c('CDPI','LiteracyRate','HealthIndex','DALYs_NCD')]), method='color', addCoef.col='black', type='upper')

```

1.7. Train / Test Split

The national time series is split into a training set (42 months: January 2021 – June 2024) and a test set (6 months: July – December 2024). Holding out the final six months enables genuine out-of-sample forecast evaluation and prevents data leakage into model parameter estimation.

```

# Step 7: Train / test split

n_test <- 6
n_train <- length(national_ts) - n_test
ts_train <- window(national_ts, end = time(national_ts)[n_train])
ts_test <- window(national_ts, start = time(national_ts)[n_train + 1])
cat('Training:', n_train, 'months | Test:', n_test, 'months')

```

2. Modelling, Diagnostics & Statistical Techniques

This section describes the three modelling approaches evaluated in the case study, the diagnostic tests applied before and after fitting, and the statistical criteria used to select the best-performing model. The modelling strategy follows a complexity-escalation principle: a linear ARIMA baseline is fitted first, then progressively more flexible nonlinear TAR specifications are evaluated.

Model	Type	Key R Functions	Key Characteristic
ARIMA(1,1,0)	Linear	auto.arima(), forecast()	Baseline; simple; assumes stationarity after differencing
TAR -- 2 Regimes	Nonlinear	setar(nthresh=1)	Captures one structural break; Low vs. High burden regimes
TAR -- 3 Regimes	Nonlinear	setar(nthresh=2)	Low/Mid/High regimes; risks over-parameterisation on short series

Table 2.1: Overview of the three modelling frameworks evaluated.

2.1. Stationarity Testing: Augmented Dickey-Fuller (ADF) Test

Any time series needs to be checked for a unit root (non-stationarity) before it can be fitted to a model. They use the Augmented Dickey-Fuller (ADF) test from the tseries package to compare H_0 , which says the series doesn't have a unit root, with H_1 , which says the series does.

- **Decision rule:** Reject H_0 if the p-value is less than 0.05 (stationary). If you don't reject, you need first-order differencing ($d=1$).
- **Expected result for log(CDPI):** p-value ≥ 0.05 at levels; p-value < 0.05 after first differencing $\Rightarrow$ confirms $d=1$ for ARIMA.

```
# Stationarity test
library(tseries) adf.test(ts_train)
# Test levels
adf.test(diff(ts_train)) # Test first differences
```

2.2. ARIMA Model: AutoRegressive Integrated Moving Average

2.2.1. Model Specification

The ARIMA(p,d,q) model is made up of three parts: AR(p) is made up of p autoregressive terms that show how past values have changed things; I(d) is made up of d -order differencing terms that make things stay the same; and MA(q) is made up of q moving-average terms that show residual autocorrelation. ARIMA(1,1,0) is the model that is automatically chosen for the log(CDPI) series:

$$\text{Equation 1. } Y_t - Y_{t-1} = c + \phi_1(Y_{t-1} - Y_{t-2}) + \epsilon_t$$

where $Y = \log(\text{CDPI})$ at time t , $\phi_1 = \text{AR coefficient } (-0.0381)$, $c = \text{intercept}$, $\epsilon_t \sim \text{i.i.d. } N(0, \sigma^2)$.

2.2.2. Fitting and Forecasting

```
# ARIMA fitting and
forecasting

library(forecast)

arima_model <-
auto.arima(ts_train, stepwise=F
ALSE, pproximation=FALSE)
summary(arima_model) #
View AIC, coefficients, RMSE
checkresiduals(arima_model) #
Ljung-Box test + residual plots
arima_fc <- forecast(arima_model, h=6) # 6-month forecast
plot(arima_fc) # Forecast fan chart
lines(ts_test, col='red') # Overlay actuals
```

Residual diagnostics: `checkresiduals()` automatically plots residuals, their ACF, and performs the **Ljung-Box test** to confirm no remaining autocorrelation in residuals ($p > 0.05$ desired). Residuals should be centred around zero with constant variance.

2.3. Two-Regime TAR Model (TAR-2)

2.3.1. Model Specification

The **Threshold Autoregressive (TAR)** model (Tsay, 2010) is a piecewise linear model that partitions the series into regimes separated by a threshold θ estimated from the data. With one threshold ($n_{\text{thresh}}=1$), two regimes emerge:

$$y_t = \mu_1 + \phi_1 y_{t-1} + \epsilon_t \text{ if } y_{t-1} \leq \theta \text{ [Regime 1: Low burden]} \\ y_t = \mu_2 + \phi_2 y_{t-1} + \epsilon_t \text{ if } y_{t-1} > \theta \text{ [Regime 2: High burden]}$$

Estimated threshold: $\theta = 3.847$ (log scale). 80.3% of observations fall in Regime 1 (low-burden states); 19.7% in Regime 2 (high-burden states — Bihar, UP, Rajasthan, etc.).

2.3.2. Fitting and Forecasting

```
# Two-regime TAR model
library(tsDyn)
```

```
tar_model2 <- setar(ts_train, m=1, thDelay=0, nthresh=1,
model='TAR') summary(tar_model2) # Threshold, coefficients, AIC
plot(residuals(tar_model2)) # Residual plot
tar2_fc <- predict(tar_model2, n.ahead=6) # 6-step forecast
```

2.4. Three-Regime TAR Model (TAR-3)

The three-regime TAR introduces two thresholds ($\theta_1 = 3.74$, $\theta_2 = 4.01$), creating Low / Mid / High burden regimes. While theoretically richer, this specification risks **over-parameterisation** on the 42-observation training set, yielding inferior out-of-sample accuracy compared with TAR-2.

Regime	% Obs.	Fitted Equation	Representative States
Low ($y \leq 3.74$)	42.1%	$0.0592 + 0.9938 * y(t-1)$	Prosperous states; Kerala, Tamil Nadu, Goa
Mid ($3.74 < y \leq 4.01$)	26.8%	$0.6021 + 0.9401 * y(t-1)$	Transitioning states
High ($y > 4.01$)	31.1%	$0.1974 + 0.9811 * y(t-1)$	High-burden states; Bihar, UP, Madhya Pradesh

Table 2.2: Three-regime TAR model structure and regime characteristics.

```
# Three-regime TAR model
tar_model3 <- setar(ts_train, m=1, thDelay=0, nthresh=2,
model='TAR') summary(tar_model3) # Two thresholds, three sets
of coefficients tar3_fc <- predict(tar_model3, n.ahead=6) # 6-step
forecast
```

2.5. Model Diagnostics

All three models are subjected to the same diagnostic battery to ensure model assumptions are met and outputs are reliable.

Diagnostic	R Code	Pass Criterion
Residual Plot	<code>plot(residuals(model))</code>	Residuals centred around zero with constant variance => no systematic bias
Ljung-Box Test	<code>Type='Ljung-Box', lag=10, residuals = Box.test(residuals)</code>	$p > 0.05$ => no remaining autocorrelation; model has captured serial structure

ACF of Residuals	<code>acf(residuals(model))</code>	All lags within confidence bands => white noise residuals confirmed
Normality (Shapiro-Wilk)	<code>shapiro.test(residuals(model))</code>	$p > 0.05$ => residuals approximately normal (supports inference validity)
Fitted vs. Actual Plot	<code>plot(fitted, actual)</code>	Visual alignment confirms in-sample fit quality

Table 2.3: Diagnostic tests applied to all three models.

2.6. Model Evaluation Criteria

Two complementary metrics are used to compare model performance, reflecting both **forecast accuracy** and **model parsimony**:

Metric	Formula	Interpretation	R Function
RMSE (Root Mean Square Error)	$\sqrt{\text{mean}((\text{actual} - \text{predicted})^2)}$	Measures average magnitude of forecast errors on the held-out 6-month test set. Lower RMSE => better out-of-sample accuracy. Penalises large errors quadratically.	<code>Metrics::rmse(actual, predicted)</code>
AIC (Akaike Information Criterion)	$-2 \cdot \log \text{Lik}(\text{model}) + 2 \cdot k$	Balances goodness-of-fit against model complexity (k = number of estimated parameters). Lower AIC => better model. Prevents over-fitting by penalising extra parameters.	<code>AIC(model)</code>

Table 2.4: Model evaluation metrics used for comparative assessment.

```
# Model evaluation -- RMSE and AIC comparison
library(Metrics)
actual_test <- as.numeric(ts_test)
cat('ARIMA RMSE:', rmse(actual_test,
  arima_preds), ' AIC:', AIC(arima_model))
cat('TAR-2 RMSE:', rmse(actual_test,
  tar2_preds), ' AIC:', AIC(tar_model2))
cat('TAR-3 RMSE:', rmse(actual_test,
  tar3_preds), ' AIC:', AIC(tar_model3))
```

Expected Model Performance Results

Model	RMSE	AIC	Rank
ARIMA(1,1,0)	0.0218	-4,859.16	3rd
TAR -- 3 Regimes	0.0962	-8,118.44	2nd
TAR -- 2 Regimes	0.0134	-8,214.70	1st (Best)

3. R Packages and Pre-defined Functions

The case study is implemented entirely using **R (FLOSS)**. All required packages are available on CRAN and are automatically installed by the project's R script. The table below provides a comprehensive reference for every package and key function used, organised by analytical purpose.

3.1. Automatic Package Installation (Portable Approach)

To ensure the script runs on any machine without manual setup, the following pattern is used at the top of every R or Rmd file. It installs only the packages not already present, then loads all of them.

```
# Portable auto-install pattern
required_packages <- c( "tidyverse",
  "forecast", "tsDyn", "tseries", "zoo",
  "Metrics", "knitr", "kableExtra", "corrplot",
  "scales")

new_pkgs <- required_packages[ !(required_packages %in% installed.packages()["Package"])
]
if (length(new_pkgs) > 0) {
  install.packages(new_pkgs, repos = "https://cloud.r-project.org")
}
invisible(lapply(required_packages, library, character.only = TRUE))
```

3.2. Complete Package and Function Reference

Package	Function(s)	Category	Purpose in This Case Study
base R	read.csv()	Data I/O	Import the CDPI CSV dataset using a relative path
base R	as.Date(), as.numeric()	Data I/O	Parse date strings and coerce character columns to numeric
base R	is.na(), sum(is.na())	Pre-processing	Detect and count missing values in each column
base R	log()	Pre-processing	Apply natural log transformation to stabilise variance

base R	ts(), window()	Time Series	Create ts objects; extract training/test windows
base R	diff()	Time Series	First-difference the series to test/achieve stationarity
base R	acf(), pacf()	Diagnostics	Plot autocorrelation and partial autocorrelation functions
base R	Box.test()	Diagnostics	Der Ljung-Box-Test überprüft, ob Restautokorrelation vorhanden ist.
base R	shapiro.test()	Diagnostics	Der Shapiro-Wilk-Test überprüft, ob die Reste normal verteilt sind.

base R	AIC()	Model Evaluation	Compute Akaike Information Criterion for any fitted model
base R	zip()	Output	Create ZIP archive of the data directory for submission
zoo	na.approx()	Pre-processing	Linear interpolation to fill missing CDPI values (rule=2 extends boundaries)
zoo	na.locf(), na.spline()	Pre-processing	Alternative imputation: last-obs-carried-forward; spline interpolation
dplyr (tidyverse)	group_by() > summarise()	Data Manipulation	Aggregate the 28-state panel to a national monthly mean series
dplyr (tidyverse)	filter(), arrange(), mutate()	Data Manipulation	Subset rows, sort by date, add transformed columns
ggplot2 (tidyverse)	ggplot() + geom_line()	Visualisation	Time series plot of log(CDPI) by state, coloured by regime
ggplot2 (tidyverse)	geom_boxplot()	Visualisation	CDPI distribution comparison across TAR-2 regimes
ggplot2 (tidyverse)	geom_point() + geom_smooth()	Visualisation	Scatter plot of CDPI vs. literacy rate with regression smoother

forecast	auto.arima()	Modelling	Automatically select ARIMA(p,d,q) order by minimising AIC (stepwise=FALSE for exhaustive search)
forecast	forecast(model, h=6)	Forecasting	Generate h-step-ahead point forecasts with 80/95% prediction intervals
forecast	fitted(arima_model)	Diagnostics	Extract in-sample fitted values from the ARIMA model
forecast	checkresiduals()	Diagnostics	One-function residual diagnostic: plot, ACF, and Ljung-Box test
tsDyn	setar(nthresh=1)	Modelling	Fit two-regime Self-Exciting TAR model; estimates threshold automatically
tsDyn	setar(nthresh=2)	Modelling	Fit three-regime TAR model with two estimated thresholds
tsDyn	predict(tar_model, n.ahead=6)	Forecasting	Generate multi-step-ahead forecasts from fitted TAR model
tseries	adf.test()	Diagnostics	Augmented Dickey-Fuller test for unit root / stationarity
Metrics	rmse(actual, predicted)	Model Evaluation	Root Mean Square Error between forecast and held-out actuals
corrplot	corrplot(cor(df[,cols]))	Visualisation	Colour-coded correlation heatmap of CDPI vs. socioeconomic determinants
knitr	kable(df)	Reporting	Produce formatted tables in HTML and PDF output from R Markdown
kableExtra	kable_styling(), row_spec()	Reporting	Apply Bootstrap styling, colour row highlighting, and column formatting
scales	comma(), percent()	Visualisation	Format axis tick labels with thousands separators or percentage notation

grid	grid.text(), grid.rect()	Output	Base-R graphics primitives for building the dataset description PDF
------	-----------------------------	--------	---

Table 3.1: Complete R package and function reference for the Chronic Disease Case Study.

3.3. Recommended Package Versions

The case study was developed and tested with the following R and package versions. The `sessionInfo()` function at the end of the R Markdown document automatically records the exact versions used on any machine.

Package	Recommended Version	Primary Use
R	4.3.3 (2024-02-29)	Core language runtime
forecast	8.21+	auto.arima(), forecast()
tsDyn	11.0+	setar(), predict.setar()
tseries	0.10-54+	adf.test()
zoo	1.8-12+	na.approx()
Metrics	0.1.4+	rmse()
ggplot2	3.4+	All visualisations
dplyr	1.1+	group_by(), summarise()
corrplot	0.92+	corrplot()
knitr	1.45+	kable()
kableExtra	1.3.4+	kable_styling()
grid	4.3.3 (base)	PDF graphics primitives

Table 3.2: Recommended R and package versions for full reproducibility.

3.4. Capturing the Runtime Environment

Including `sessionInfo()` as the final chunk of every R Markdown document is a best practice for reproducible research. It records the exact R version, operating system, and all loaded package versions, enabling any collaborator to replicate results precisely.

```
# Always include this as the final chunk in your .Rmd
sessionInfo()

# Example output (abbreviated):
# R version 4.3.3 (2024-02-29)
```

```
# Platform: x86_64-pc-linux-gnu (64-bit) # Loaded packages:
# forecast 8.21, tsDyn 11.0, tseries 0.10-54, zoo 1.8-12,
# Metrics 0.1.4, ggplot2 3.4.4, dplyr 1.1.4, corrplot 0.92
```

Methodology Summary

1. **Pre-processing:** read.csv (relative path), type conversion, na.approx imputation (zoo), log transformation, dplyr aggregation, ts() object, train/test split (42/6 months), EDA (ggplot2, acf, pacf, corrplot).
2. **Modelling:** ADF stationarity test (tseries), ARIMA(1,1,0) via auto.arima (forecast), TAR-2 via setar(nthresh=1) (tsDyn), TAR-3 via setar(nthresh=2) (tsDyn), residual diagnostics (checkresiduals, Ljung-Box, Shapiro-Wilk).
3. **Evaluation:** RMSE via Metrics::rmse(), AIC via base-R AIC(), 6-month out-of-sample forecast comparison, formatted results tables via knitr::kable() + kableExtra.

Best Model: TAR-2 Regime (RMSE = 0.0134, AIC = -8,214.70) confirms nonlinear regime-switching dynamics in India's NCD burden.